中国研究生操作系统开源创新大赛

项目说明书

作品名称：面向智能机器人的 AIOS

参赛队伍：Xenoblade

完成日期：2026 年 10 月

目 录

第 1 章 绪 论 **1**
1.1 目的与意义 1
1.2 项目背景 1
第 2 章 技 术 理 论 **4**
2.1 设计思想 4
2.2 技术路线 4
2.3 代码原创说明 4
第 3 章 项目介绍 **5**
3.1 功能介绍 5
3.1.1 xxx 模块 5
3.1.2 xxx 模块 5
3.2 软件界面 （可选） 5
3.2.1 X 主要模块界面 5
第 4 章 实验测试 **6**
4.1 测试说明 6
4.2 测试结果 6
第 5 章 实现难点说明 **7**
第 6 章 总 结 **8**
参 考 **9**

第 1 章　绪 论

1.1　目的与意义

伴随具身智能与 AI 大模型的快速演进，机器人正由受控环境走向开放而复杂的真实场景，其应用普遍包含高频视觉感知、实时决策与运动控制，对系统的实时性与资源利用率提出了严苛要求。这类系统通常把模型、调度、驱动与供电等模块集成在同一块 SoC（片上系统）内，操作系统既要不断向加速器投喂数据，又要协调多核之间的处理器时间分配。当前主流做法是在 Linux 等通用操作系统上搭建，其成熟的驱动与软件生态虽然便利，却也引入冗余服务、较长的执行路径、复杂的调度策略与较大的内存开销，在边缘算力受限时容易转化为端到端延迟与抖动上的负担。如果改由 Rust 编写、具备组件化特征的内核来承载同样的系统，内存安全与模块化程度可望提升，但必须以通用负载性能不受损为条件。本项目所采用的载体是一台网球拾取机器人，其程序来自 `chenlongos/akars`，已经以 Rust 实现，能够同时运行在 Linux 与 StarryOS 之上。该机器人的执行回路由追球、抓取、回桶、寻桶、趋近和投放六个状态构成，评价指标取相机成帧到执行器指令之间的帧到指令延迟，与竞赛官方口径一致。

项目工作沿应用层与系统层两条线索推进。在应用层，目标是让这台机器人在 StarryOS 上完整跑起来，并对推理链路加以优化，借助取球循环的组织方式与数据布局的调整，把帧到指令延迟压缩到与 Linux 相当的水平，同时补齐 TPU 推理所依赖的关键外设，使推理通路真正贯通。在系统层，则把 StarryOS 当作一个通用操作系统来对待，在调度、系统调用、内存、频率这些与具体应用无直接绑定的子系统上，与 Linux 逐项对照。机器人因而只是被选中的负载之一，真正被检验的是操作系统本身的通用能力。

为使两侧结论具备可比性，评测口径需要预先统一：各子系统配套专属的基准套件，并各自设立独立的 Linux 基线；双方执行的是同一份符合 Linux ABI 的可执行文件，因此同一负载在两侧呈现出的差别，只会来自操作系统本身。下文先交代机器人应用，随后逐一讨论 xxx、xxx、xxx 等系统层子系统。凡属对等比较的子系统，均以相应的 Linux 基线为参照并说明所做的改动；至于 xxx 一类以功能使能或仿真为目的的工作，则不计入对等结论。

1.2　项目背景

赛题「面向智能机器人的 AIOS」要求把开源机器人的操作系统内核从 C 语言实现的 Linux 整体迁移到 以 Rust 编写的组件化内核，在内核之上补齐系统调用、文件接口与 NPU/USB/UART/摄像头等关键外设驱动， 打通 NPU 推理链路，并在端到端延迟、启动速度、推理帧数与资源、能耗比等维度超过同机 Linux 基线。 赛程分三步走：先在开发板上把内核启动起来并完成模型加载，再补齐 USB 感知与串口电机驱动、完成 网球的拾取，最后与 Linux 基线逐项对标做性能优化。本项目落在第三步，交付的是一份可复现的对标 结论，以及为得到它而做的内核改动。

硬件载体是辰龙 AKA-00 教育机器人，主控为 SG2002：应用核是 RISC-V C906 @ 1 GHz（与 ARM Cortex-A53 二者取一，由管脚选择），另有 C906 @ 700 MHz 协处理器， 片上 TPU 的 INT8 算力为 1 TOPS，板载 DDR 256 MiB。机器人程序是 `akars`，由同名的 C 语言版本移植为 Rust 二进制，链路为 USB 摄像头取流、MJPEG 解码与 letterbox、 YOLOv8 的 CVI 运行时推理、串口底盘与串口机械臂，六个状态构成拾球闭环。它运行在 musl 根文件 系统之上，依赖厂

商预编译的 `libcviruntime` 与 `libcvikernel`，并按 Linux 的 `ioctl` 与 `mmap` 约定 访问相机与 TPU。算力与内存受限、软件栈以 Linux 二进制形式固化、闭环对延迟敏感，这三条约束 共同划定了「换内核」一事的可行边界。

Linux 成为这类机器人的默认底座，理由很实在：驱动与中间件完备，摄像头、USB、文件系统与网络 开箱可用；ABI 稳定，厂商以二进制分发的推理运行时可原样运行；工具链与调试手段成熟，开发门槛低。代价来自同一处，也就是它的通用性。为兼容各种场景，系统里常驻着一批与任务无关的服务与抽象，一次调用要穿过比实际需要更长的路径；调度器在通用负载之间做公平取舍，内存子系统的许多假设在 大内存机器上成立、在小内存设备上则是额外开销。这些开销在桌面与服务器上被充裕的算力吸收，在 边缘受限时直接表现为端到端延迟与抖动。本项目的实测支持这一判断：厂商 Linux 从上电到首帧推理 耗时 34.23 s，其中内核初始化只占 1.2 s，其余 31.99 s 几乎都花在用户态的初始化与服务上；同一块板上，StarryOS 的内核初始化是 9.01 s， 用户态只要 4.2 s，总的 14.66 s 反而更短。通用底座把相当一部分 开销放在了内核之外，而它省下来的内核工作时间有限。

不同系统形态各有取舍，对照见表 1。

系统形态	代表	优势	在本题约束下的局限
C 语言宏内核	厂商 Linux 5.10	驱动与中间件完备，USB、摄像头、文件系统与网络开箱可用；ABI 稳定，厂商以二进制分发的 NPU 运行时可直接运行。	通用性带来冗余服务与较长的执行路径；空系统调用、缺页、块设备等入口开销高；常驻内存占用大。
实时内核	FreeRTOS、RT-Thread、Zephyr、NuttX	确定性好、体积小、上电即用，适合承担确定性感知前端。	没有 Linux ABI 与完整的进程、文件语义，厂商预编译的相机与 NPU 运行时无法复用； SG2002 的小核固件本身也不含 USB 与 TPU 驱动。
组件化 unikernel	ArceOS	能力以 `crate` 组合，裁剪自由度最大，是 StarryOS 的组件底座。	面向单地址空间应用，不带进程、信号与 Linux 兼容系统调用，无法直接承载已经编译好的机器人程序。
Rust 宏内核	StarryOS（TGOSKits）	在组件层之上补齐 Linux 系统调用、进程与信号；内存安全、模块化，可直接运行符合 Linux ABI 的可执行文件。	性能与稳定性尚未对齐 Linux，系统调用入口、内存与启动仍有明确差距。

表 1: 主流系统形态在本题约束下的取舍

把上表收拢成一句：Linux 的长处几乎都来自「通用」，短板也几乎都来自「通用」；实时内核与组件化 unikernel 则各自用放弃兼容性换来确定性与可裁剪性。赛题给定的条件同时要求直接运行厂商预编译的 机器人二进制、覆盖 USB/UART/摄像头/TPU 四类外设、并与 Linux 基线逐项对标，能满足这三条的形态 只有「Rust 组件化内核 + Linux ABI 兼容」，也就是 StarryOS 所在的位置。

上游 TGOSKits 已经提供了这个起点：一个含 184 个 workspace 成员的 三系统工作区（ArceOS、StarryOS、Axvisor）、4 种 64 位架构，以及可复用的组件与 驱动，其中包含 SG2002 的 TPU 与它依赖的 ION 分配器、USB 子系统与串口。板卡侧，`board-aka-00-sg2002` 一条配置下已登记启动、`lsusb`、`libuvc` 取流、WiFi 压测与 `tennis-yolo` 检测 等用例；其中 `tennis-yolo` 用例要求板上打印 `STARRY_AKA00_TENNIS_DETECT_OK`，说明「模型能被加载 并识别出网球」这一环在上游已经打通。缺口在别处。把内核换成 StarryOS 之后，空系统调用的开销 相对 Linux 高出 26.7 倍，模型加载耗时 2.99 s 对 Linux 的 0.07 s，取球过程的峰值内存 19316 kB 对 Linux 的 7604 kB。 系统层因而仍有明确的优化空间，这正是本项目介入的位置。

本作品的新颖之处有三。

第一是问题域。赛题给出两台参考平台，本队落在 SG2002 一侧：它只有一颗可运行 Linux 的 C906 应用核，没有 ARM 大小核，也没有可供内核使用的调频调压设施与 PMU，RK3588 上那套成熟的大小核调度与 DVFS 路径在这里无法复制。可优化的对象随之改变——系统调用入口的固定开销、缺页与内存映射、块设备的 写放大与 `fsync`、上电到首帧的启动链路，以及网络与 WiFi 的稳定性，成为左右端到端表现的主要因素。

第二是评测口径。既有工作多以「优化前后」自证，本作品要求每条结论都带对照：同一块板、同一份符合 Linux ABI 的可执行文件，在 StarryOS 与厂商 Linux 两条基线上各测一次；在 StarryOS 一侧再做四分支 横比，即本分支、报告团队的集成分支与两条上游分支。四分支横比把「这条改动是否真的有效」与 「这条改动是否已经在上游生效」分开，避免把移植来的既有能力记成新的贡献。

第三是贡献归属。报告中每条优化都带 `ours` 与 `ref` 标记，区分本队独立完成的改动与对参考报告团队 成果的引用和复现；以功能使能或仿真为目的的工作不计入对等结论。工作按三条线组织：评测框架与 探针模块把「可测」变成「测得准」，内核缺陷修复处理已知的挂起、容量墙与失效场景，独立优化则在 前两条线的结果上逐项对标 Linux。三条线共同的立场是，机器人只是被选中的负载之一，真正被检验的 是操作系统的通用能力。

第 2 章　技 术 理 论

2.1　设计思想

2.2　技术路线

2.3　代码原创说明

第 3 章　项目介绍

3.1　功能介绍

3.1.1　xxx 模块

3.1.2　xxx 模块

3.2　软件界面 （可选）

3.2.1　X 主要模块界面

第 4 章　实验测试

4.1　测试说明

4.2　测试结果

第 5 章　实现难点说明

第 6 章　总 结

参考